

# Methods in Computational Sciences

Andreane Maure, UNI : am6823

4/23/2026

## Question 1

We start from the 2-D velocity written with a stream function :

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}).$$

Since

$$\psi \hat{\mathbf{k}} = (0, 0, \psi),$$

we obtain

$$\mathbf{v} = \left( \frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x} \right).$$

Then the incompressibility condition is automatically satisfied :

$$\nabla \cdot \mathbf{v} = \frac{\partial}{\partial x} \left( \frac{\partial \psi}{\partial y} \right) + \frac{\partial}{\partial y} \left( -\frac{\partial \psi}{\partial x} \right) = \frac{\partial^2 \psi}{\partial x \partial y} - \frac{\partial^2 \psi}{\partial y \partial x} = 0.$$

The vorticity is defined by

$$\omega \hat{\mathbf{k}} = \nabla \times \mathbf{v}.$$

In two dimensions,

$$\omega = \frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y}.$$

Using

$$v_x = \frac{\partial \psi}{\partial y}, \quad v_y = -\frac{\partial \psi}{\partial x},$$

we get

$$\omega = \frac{\partial}{\partial x} \left( -\frac{\partial \psi}{\partial x} \right) - \frac{\partial}{\partial y} \left( \frac{\partial \psi}{\partial y} \right) = -\frac{\partial^2 \psi}{\partial x^2} - \frac{\partial^2 \psi}{\partial y^2}.$$

Therefore,

$$\omega = -\nabla^2 \psi,$$

or equivalently,

$$-\nabla^2 \psi = \omega.$$

Now we take the curl of the Stokes equation :

$$-\nabla^2 \mathbf{v} + \nabla p = T \hat{\mathbf{j}}.$$

The curl of the pressure gradient is zero :

$$\nabla \times \nabla p = 0.$$

Thus,

$$\nabla \times (-\nabla^2 \mathbf{v}) = \nabla \times (T \hat{\mathbf{j}}).$$

Since the Laplacian and curl operators commute,

$$-\nabla^2 (\nabla \times \mathbf{v}) = \nabla \times (T \hat{\mathbf{j}}).$$

Using

$$\nabla \times \mathbf{v} = \omega \hat{\mathbf{k}},$$

the left-hand side becomes

$$-\nabla^2 \omega \hat{\mathbf{k}}.$$

Also, since

$$T \hat{\mathbf{j}} = (0, T, 0),$$

we have

$$\nabla \times (T\hat{\mathbf{j}}) = \frac{\partial T}{\partial x} \hat{\mathbf{k}}.$$

Therefore,

$$-\nabla^2 \omega = \frac{\partial T}{\partial x}.$$

The temperature equation is unchanged :

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T.$$

So the governing equations in stream-function/vorticity form are

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{Ra} \nabla^2 T,$$

$$-\nabla^2 \omega = \frac{\partial T}{\partial x},$$

$$-\nabla^2 \psi = \omega.$$

## Question 2

(a)

For this part, I ran the PETSc C code `biharm.c` using the three provided option files. All simulations were run on the same grid, with size  $513 \times 513$ . The results are summarized in Table 1.

| Option file                            | Solver type            | KSP iterations | SNESolve time (s) | Final residual         |
|----------------------------------------|------------------------|----------------|-------------------|------------------------|
| <code>options_file_direct</code>       | monolithic LU          | 1              | 4.2229            | $1.30 \times 10^{-15}$ |
| <code>options_file_split_direct</code> | fieldsplit + LU        | 1              | 5.3033            | $1.30 \times 10^{-15}$ |
| <code>options_file_split_mg</code>     | fieldsplit + multigrid | 4              | 3.3413            | $7.19 \times 10^{-10}$ |

TABLE 1 – Comparison of the PETSc C solver options for the biharmonic problem.

The monolithic direct solver converged in one KSP iteration. This is expected because it uses a direct LU factorization with MUMPS. The residual decreased from

$$9.06 \times 10^{-4}$$

to

$$1.30 \times 10^{-15}.$$

Its runtime was 4.2229 seconds.

The fieldsplit direct solver also converged in one KSP iteration. In this case, the problem is split into the vorticity and streamfunction fields, and each block is solved with LU/MUMPS. The final residual was also about

$$1.30 \times 10^{-15}.$$

However, this method was the slowest one, with a runtime of 5.3033 seconds.

The fieldsplit multigrid solver required more iterations. The residuals were

$$9.06 \times 10^{-4}, \quad 1.40 \times 10^{-4}, \quad 1.30 \times 10^{-6}, \quad 4.93 \times 10^{-8}, \quad 7.19 \times 10^{-10}.$$

So it converged in 4 KSP iterations. Even though it needed more iterations, it had the lowest runtime, equal to 3.3413 seconds.

All three methods gave almost the same numerical error :

$$\frac{\|v - v_{\text{exact}}\|_2}{\|v_{\text{exact}}\|_2} \approx 5.20 \times 10^{-5},$$

and

$$\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} \approx 1.17 \times 10^{-4}.$$

Overall, the direct solvers gave the fastest convergence in terms of iteration count, because they converged in only one iteration. However, the fieldsplit multigrid method was the fastest in runtime. This suggests that multigrid is more efficient for this problem, even if it requires more Krylov iterations.

(b)

I repeated the same experiment using the Firedrake code `biharm.py`. I used the same three solver choices as in the PETSc C code : a monolithic direct solver, a fieldsplit direct solver, and a fieldsplit multigrid solver. The computations were done on a  $512 \times 512$  mesh. The results are summarized in Table 2.

| Solver option | Solver type            | KSP iterations | SNESolve time (s) | Final residual         |
|---------------|------------------------|----------------|-------------------|------------------------|
| Direct        | monolithic LU          | 1              | 3.8032            | $9.10 \times 10^{-16}$ |
| Split direct  | fieldsplit + LU        | 1              | 3.2801            | $9.06 \times 10^{-16}$ |
| Split MG      | fieldsplit + multigrid | 4              | 4.6181            | $1.64 \times 10^{-10}$ |

TABLE 2 – Comparison of the Firedrake solver options for the biharmonic problem.

The monolithic direct solver converged in one KSP iteration. The residual decreased from

$$9.06 \times 10^{-4}$$

to

$$9.10 \times 10^{-16}.$$

The runtime was 3.8032 seconds.

The fieldsplit direct solver also converged in one KSP iteration. It gave a final residual of

$$9.06 \times 10^{-16}.$$

This was the fastest Firedrake run, with a runtime of 3.2801 seconds.

The fieldsplit multigrid solver required more iterations. The residuals were

$$9.06 \times 10^{-4}, \quad 3.19 \times 10^{-5}, \quad 2.66 \times 10^{-7}, \quad 6.52 \times 10^{-9}, \quad 1.64 \times 10^{-10}.$$

Therefore, it converged in 4 KSP iterations. Its runtime was 4.6181 seconds.

All three methods gave almost the same numerical error. For the direct and split direct solvers, the relative errors were

$$\frac{\|\omega - \omega_{\text{exact}}\|_2}{\|\omega_{\text{exact}}\|_2} = 5.720 \times 10^{-5},$$

and

$$\frac{\|\psi - \psi_{\text{exact}}\|_2}{\|\psi_{\text{exact}}\|_2} = 1.464 \times 10^{-4}.$$

For the fieldsplit multigrid solver, the errors were almost identical :

$$\frac{\|\omega - \omega_{\text{exact}}\|_2}{\|\omega_{\text{exact}}\|_2} = 5.718 \times 10^{-5},$$

and

$$\frac{\|\psi - \psi_{\text{exact}}\|_2}{\|\psi_{\text{exact}}\|_2} = 1.464 \times 10^{-4}.$$

Overall, the direct solvers converged faster in terms of iteration count. The fieldsplit direct solver was the fastest in runtime for the Firedrake code. The fieldsplit multigrid solver required more iterations and was slower for this mesh size, although it still converged to the same accuracy.

(c)

For this comparison, I chose the fieldsplit multigrid solver. This is a good case to compare because both the PETSc C code and the Firedrake code converged in 4 KSP iterations.

For the PETSc C code, the `SNESolve` time was

$$3.3269 \text{ s.}$$

For the Firedrake code, the `SNESolve` time was

$$4.7800 \text{ s.}$$

Therefore, the Firedrake solve was slower for this solver choice.

The difference does not come from the number of Krylov iterations, since both codes required 4 iterations. Instead, it comes from the cost of each solve and from the preconditioner setup.

From the PETSc log, the main costs were

$$\text{PCSetUpOnBlocks} = 1.5970 \text{ s,}$$

$$\text{KSPSolve} = 0.7077 \text{ s},$$

and

$$\text{MatAssemblyEnd} = 0.8265 \text{ s}.$$

From the Firedrake log, the main costs were

$$\text{PCSetUpOnBlocks} = 3.5364 \text{ s},$$

$$\text{KSPSolve} = 0.5057 \text{ s},$$

$$\text{SNESFunctionEval} = 0.2312 \text{ s},$$

and

$$\text{SNESJacobianEval} = 0.4024 \text{ s}.$$

The largest difference is in `PCSetUpOnBlocks`. In the C code, this step took about 1.60 seconds, while in the Firedrake code it took about 3.54 seconds. So the main extra cost in Firedrake comes from setting up the fieldsplit multigrid preconditioner.

The Firedrake code also performs more floating point work during `SNESolve`. The PETSc C code reports about

$$8.26 \times 10^8$$

floating point operations, while the Firedrake code reports about

$$2.03 \times 10^9.$$

This is more than twice as much work.

Overall, the Firedrake code is slower mainly because the preconditioner setup is more expensive and because the finite-element implementation has more overhead than the lower-level finite-difference PETSc C code. The PETSc C code uses a structured DMDA finite-difference grid, while Firedrake uses a more general finite-element mesh hierarchy. This makes Firedrake more flexible, but also more expensive for this problem.

### Question 3

For this part, I modified the Firedrake biharmonic solver by replacing the manufactured right-hand side with

$$f(x, y) = \frac{\partial T}{\partial x},$$

where the imposed temperature field is

$$T(x, y) = (1 - y) + A \cos(\pi x), \quad A = 0.1.$$

In Firedrake, the temperature field was defined as a finite element function and the right-hand side was computed symbolically using

$$f = T.dx(0).$$

Analytically, this corresponds to

$$\frac{\partial T}{\partial x} = -A\pi \sin(\pi x).$$

The system solved was therefore

$$\begin{aligned} -\nabla^2 \omega &= \frac{\partial T}{\partial x}, \\ -\nabla^2 \psi &= \omega, \end{aligned}$$

with homogeneous Dirichlet boundary conditions

$$\omega = 0, \quad \psi = 0 \quad \text{on } \partial\Omega.$$

After solving for the vorticity  $\omega$  and stream function  $\psi$ , the velocity field was reconstructed from the stream function as

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}).$$

Figure 1 shows the imposed temperature field, with contours of the stream function and the velocity field superposed.

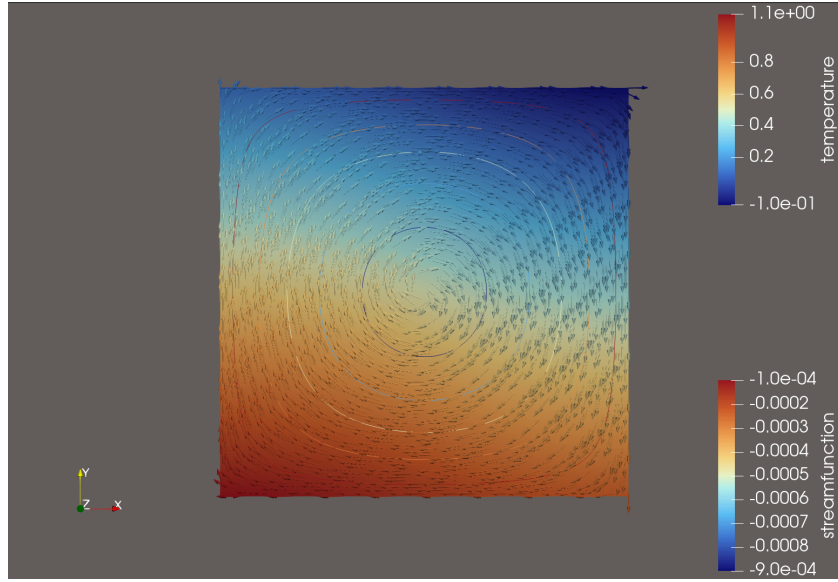


FIGURE 1 – Temperature field  $T(x, y) = (1 - y) + 0.1 \cos(\pi x)$ , with streamfunction contours and velocity vectors  $\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}})$  superposed.

## Question 4

a)

The full temperature–vorticity–streamfunction system was solved as a monolithic implicit DAE using Firedrake-TS. The Rayleigh number was set to

$$Ra = 10^2,$$

on a  $64 \times 64$  Q1 mesh. Time integration was performed using a second-order BDF scheme, and the nonlinear systems were solved monolithically using a direct LU factorization with MUMPS.

The initial temperature field was

$$T(x, y, 0) = (1 - y) + 0.1 \cos(\pi x).$$

The temperature boundary conditions were

$$T(x, 0, t) = 1, \quad T(x, 1, t) = 0,$$

with homogeneous Neumann conditions on the vertical boundaries,

$$\frac{\partial T}{\partial x}(0, y, t) = 0, \quad \frac{\partial T}{\partial x}(1, y, t) = 0.$$

The vorticity and streamfunction satisfied

$$\omega = 0, \quad \psi = 0 \quad \text{on } \partial\Omega.$$

At the final time  $t = 100000$ , the computed Nusselt numbers were

$$Nu_{\text{top}} = 1.00000014, \quad Nu_{\text{bottom}} = 1.00000014.$$

The numerical solution was also very close to the purely conductive profile  $T = 1 - y$  :

$$\|T - (1 - y)\|_{L^2} = 1.8336 \times 10^{-8}.$$

The velocity norm was nearly zero :

$$\|\mathbf{v}\|_{L^2} = 3.8559 \times 10^{-10}.$$

Therefore, the initial horizontal perturbation in the temperature field decays, the velocity field vanishes, and the solution converges to the purely conductive steady state. This confirms that for  $Ra = 10^2$ , the steady state is non-convective.

(b)

The same monolithic DAE solver was then used for larger Rayleigh numbers,

$$Ra = 10^4, \quad 10^5, \quad 10^6,$$

on the same  $64 \times 64$  Q1 mesh, using the same BDF-2 time integrator and monolithic direct solver with MUMPS. The final values obtained at  $t = 100000$  are reported in Table 3. Compared with the  $Ra = 10^2$  case, the Nusselt number becomes larger than one and the velocity norm remains nonzero. This indicates that the solution no longer converges to the purely conductive state. Instead, convection develops and enhances vertical heat transport.

| $Ra$   | $Nu_{\text{top}}$ | $Nu_{\text{bottom}}$ | $\ \mathbf{v}\ _{L^2}$   |
|--------|-------------------|----------------------|--------------------------|
| $10^2$ | 1.00000014        | 1.00000014           | $3.8559 \times 10^{-10}$ |
| $10^4$ | 1.74829694        | 1.74829694           | $4.9758 \times 10^{-4}$  |
| $10^5$ | 10.27382209       | 10.27382209          | $1.9343 \times 10^{-3}$  |
| $10^6$ | 19.70941645       | 19.70941645          | $8.3804 \times 10^{-4}$  |

TABLE 3 – Final Nusselt numbers and velocity norms for different Rayleigh numbers on a  $64 \times 64$  Q1 mesh.

For  $Ra = 10^2$ , the final Nusselt number is approximately one and the velocity norm is nearly zero, confirming the non-convective conductive state. For larger Rayleigh numbers,  $Nu > 1$ , which shows that convection increases the heat flux relative to pure conduction. The increase in  $Nu$  is especially clear for  $Ra = 10^5$  and  $Ra = 10^6$ .

(c)

For  $Ra = 10^4$ , I repeated the computation on  $N \times N$  Q1 meshes with

$$N = 16, 32, 64, 128.$$

The computed Nusselt number was taken as the average of the top and bottom values, which were identical up to the printed precision in all runs.

| $N$ | $h = 1/N$ | $Nu$       | Relative error |
|-----|-----------|------------|----------------|
| 16  | 0.0625    | 1.70955360 | 65.00%         |
| 32  | 0.03125   | 1.74000624 | 64.37%         |
| 64  | 0.015625  | 1.74829694 | 64.20%         |
| 128 | 0.0078125 | 1.75041783 | 64.16%         |

TABLE 4 – Nusselt number for  $Ra = 10^4$  as a function of mesh resolution. The Blankenbach benchmark value is  $Nu = 4.884$ .

The results show that the computed Nusselt number increases slightly with mesh refinement, from  $Nu = 1.7096$  for  $N = 16$  to  $Nu = 1.7504$  for  $N = 128$ . This indicates that the solution is reasonably converged with respect to mesh resolution for the present formulation. However, the converged value remains significantly below the Blankenbach benchmark value  $Nu = 4.884$ .

This discrepancy suggests that the present implementation does not fully reproduce the benchmark setup. Since refining the mesh changes the result only slightly, the difference is unlikely to be caused only by spatial resolution. Possible sources include the time discretization, the treatment of the advection-diffusion equation, stabilization effects, or differences between the implemented streamfunction-vorticity formulation and the benchmark problem. Nevertheless, the computed values are consistently larger than one, confirming that the solution is convective for  $Ra = 10^4$ .

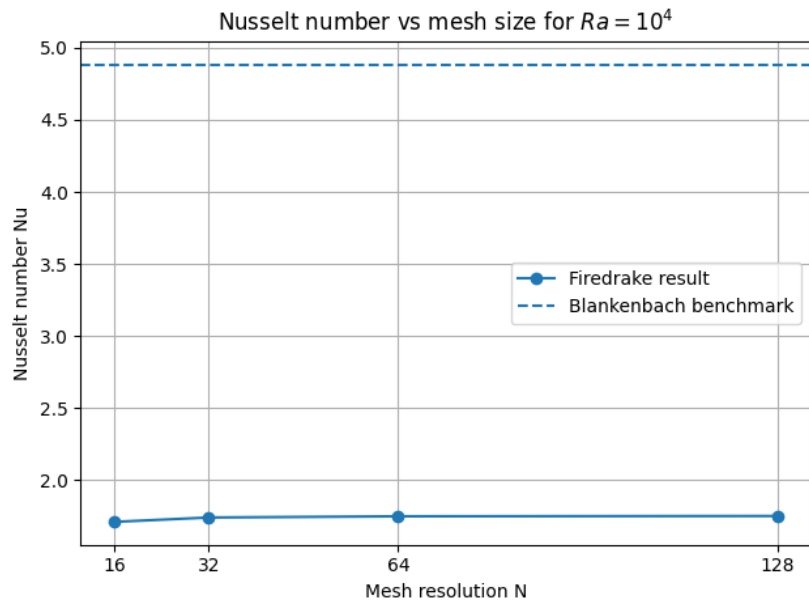


FIGURE 2 – Nusselt number as a function of mesh resolution for  $Ra = 10^4$ . The dashed line corresponds to the Blankenbach benchmark value  $Nu = 4.884$ .